

Hanoi Towers Illegal State Algorithms Performance Analysis

Luke Pepin

2025-12-03

```
# Read data from CSV
data <- read.csv("results.csv")

# Convert to factors with readable labels
data$stack_algorithm <- factor(data$stack_algorithm,
                               levels = c("dig_out", "a_star"),
                               labels = c("Dig-Out", "BFS"))
data$ground_algorithm <- factor(data$ground_algorithm,
                                levels = c("greedy", "patient"),
                                labels = c("Greedy", "Patient"))
data$corruption_rate <- factor(data$corruption_rate,
                               levels = c(0.05, 0.1),
                               labels = c("5%", "10%"))

# Rename for clarity in analysis
data$stack <- data$stack_algorithm
data$ground <- data$ground_algorithm
data$corruption <- data$corruption_rate
data$value <- data$total_moves

str(data)

## 'data.frame': 800 obs. of 15 variables:
## $ trial_id : chr "C1_T1" "C1_T2" "C1_T3" "C1_T4" ...
## $ condition_id : int 1 1 1 1 1 1 1 1 1 1 ...
## $ stack_algorithm : Factor w/ 2 levels "Dig-Out","BFS": 1 1 1 1 1 1 1 1 1 1 ...
## $ ground_algorithm : Factor w/ 2 levels "Greedy","Patient": 1 1 1 1 1 1 1 1 1 1 ...
## $ corruption_rate : Factor w/ 2 levels "5%","10%": 1 1 1 1 1 1 1 1 1 1 ...
## $ seed : int 100 200 300 400 500 600 700 800 900 1000 ...
## $ total_moves : int 45 92 45 29 29 18 66 29 49 38 ...
## $ num_regenerations : int 3 7 2 1 2 4 6 1 2 2 ...
## $ num_corruptions_occurred: int 2 6 1 0 1 3 5 0 2 1 ...
## $ final_state_valid : chr "True" "True" "True" "True" ...
## $ timeout : chr "False" "False" "False" "False" ...
## $ stack : Factor w/ 2 levels "Dig-Out","BFS": 1 1 1 1 1 1 1 1 1 1 ...
## $ ground : Factor w/ 2 levels "Greedy","Patient": 1 1 1 1 1 1 1 1 1 1 ...
## $ corruption : Factor w/ 2 levels "5%","10%": 1 1 1 1 1 1 1 1 1 1 ...
## $ value : int 45 92 45 29 29 18 66 29 49 38 ...

head(data)

## trial_id condition_id stack_algorithm ground_algorithm corruption_rate seed
```

```

## 1    C1_T1          1      Dig-Out      Greedy          5% 100
## 2    C1_T2          1      Dig-Out      Greedy          5% 200
## 3    C1_T3          1      Dig-Out      Greedy          5% 300
## 4    C1_T4          1      Dig-Out      Greedy          5% 400
## 5    C1_T5          1      Dig-Out      Greedy          5% 500
## 6    C1_T6          1      Dig-Out      Greedy          5% 600
##      total_moves num_regenerations num_corruptions_occurred final_state_valid
## 1          45          3              2              True
## 2          92          7              6              True
## 3          45          2              1              True
## 4          29          1              0              True
## 5          29          2              1              True
## 6          18          4              3              True
##      timeout   stack ground corruption value
## 1    False Dig-Out Greedy          5%    45
## 2    False Dig-Out Greedy          5%    92
## 3    False Dig-Out Greedy          5%    45
## 4    False Dig-Out Greedy          5%    29
## 5    False Dig-Out Greedy          5%    29
## 6    False Dig-Out Greedy          5%    18

```

```

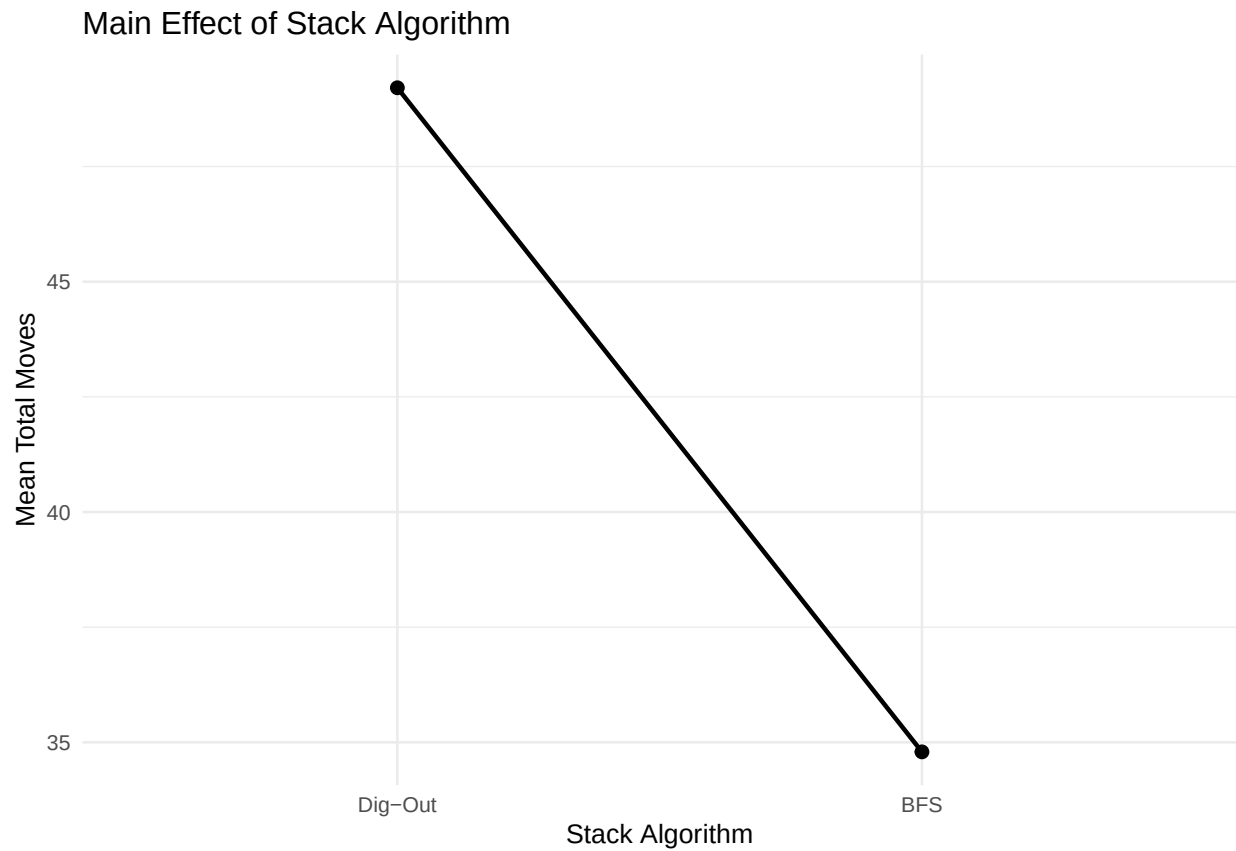
# Main effect: Stack Algorithm

```

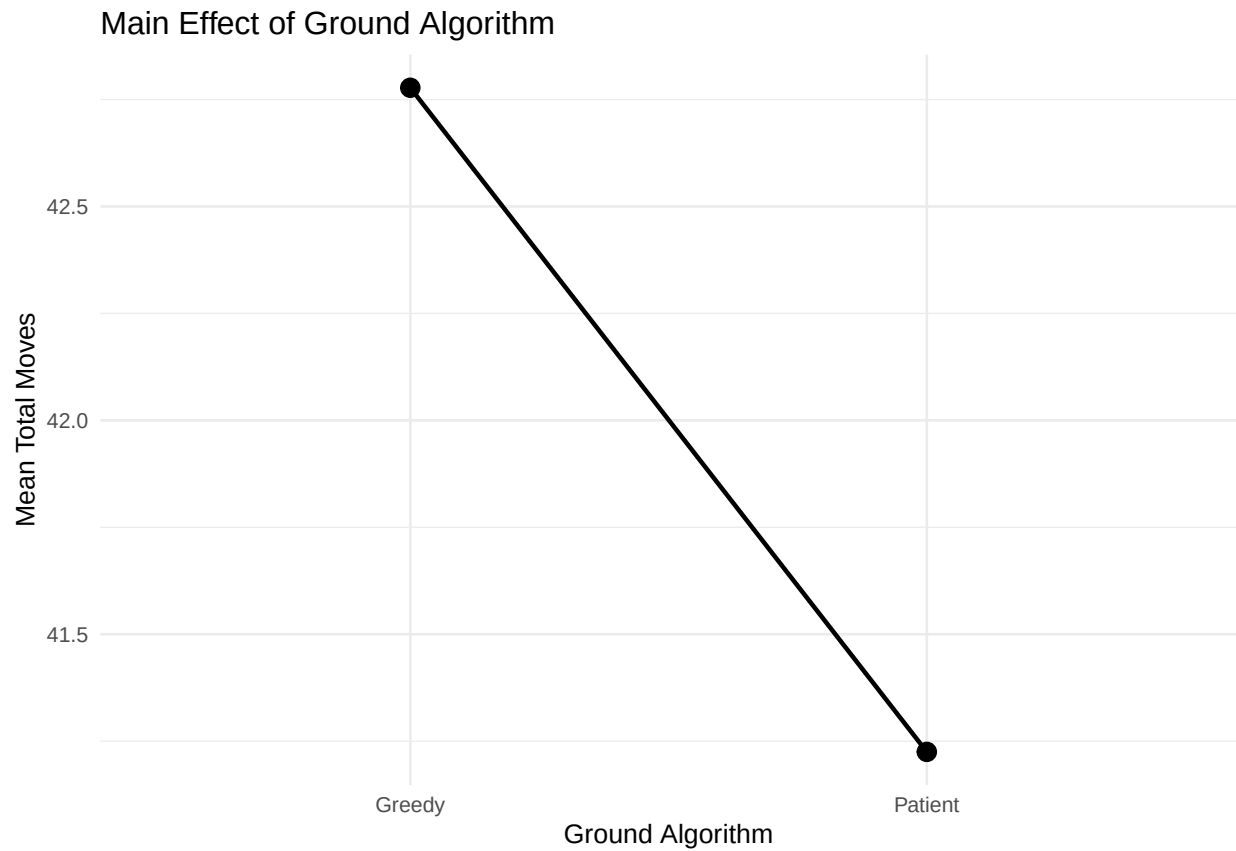
```

ggplot(data, aes(x = stack, y = value)) +
  stat_summary(fun = mean, geom = "point", size = 2) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), linewidth = 0.8) +
  labs(title = "Main Effect of Stack Algorithm",
       y = "Mean Total Moves",
       x = "Stack Algorithm") +
  theme_minimal(base_size = 10)

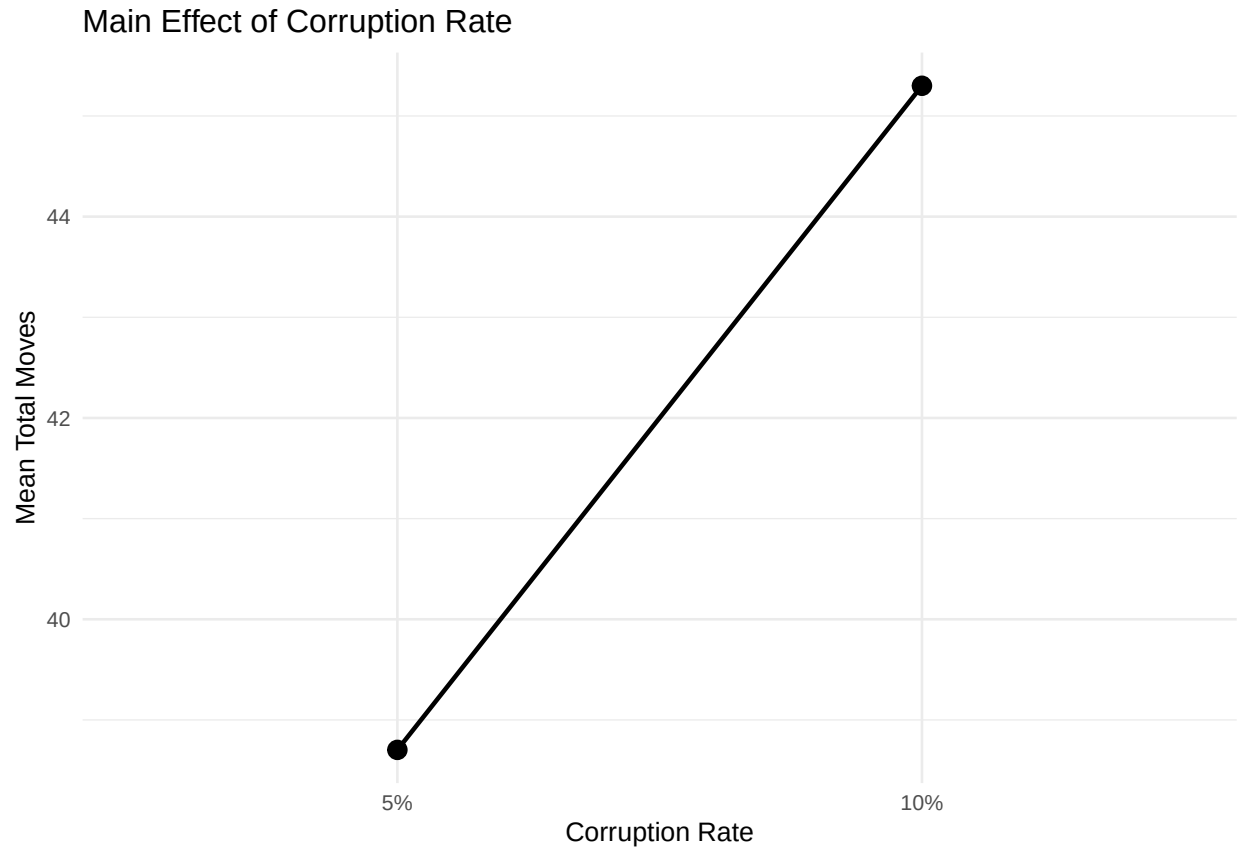
```



```
# Main effect: Ground Algorithm
ggplot(data, aes(x = ground, y = value)) +
  stat_summary(fun = mean, geom = "point", size = 3) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), linewidth = 0.8) +
  labs(title = "Main Effect of Ground Algorithm",
       y = "Mean Total Moves",
       x = "Ground Algorithm") +
  theme_minimal(base_size = 10)
```



```
# Main effect: Corruption Rate
ggplot(data, aes(x = corruption, y = value)) +
  stat_summary(fun = mean, geom = "point", size = 3) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), linewidth = 0.8) +
  labs(title = "Main Effect of Corruption Rate",
       y = "Mean Total Moves",
       x = "Corruption Rate") +
  theme_minimal(base_size = 10)
```



Calculate group means for interaction plots

```
group_means <- data %>%
  group_by(stack, ground, corruption) %>%
  summarise(
    mean_value = mean(value),
    sd = sd(value),
    n = n(),
    se = sd / sqrt(n),
    .groups = 'drop'
  )
```

```
print(group_means)
```

```
## # A tibble: 8 x 7
```

```
##   stack  ground corruption mean_value    sd     n    se
##   <fct>  <fct>   <fct>      <dbl> <dbl> <int> <dbl>
## 1 Dig-Out Greedy  5%         46.9  22.3   100  2.23
## 2 Dig-Out Greedy 10%         52.4  26.9   100  2.69
## 3 Dig-Out Patient 5%         43.7  19.7   100  1.97
## 4 Dig-Out Patient 10%        53.8  33.4   100  3.34
## 5 BFS     Greedy  5%         32.1  14.9   100  1.49
## 6 BFS     Greedy 10%         39.7  23.6   100  2.36
## 7 BFS     Patient 5%         32.1  18.4   100  1.84
## 8 BFS     Patient 10%        35.3  23.5   100  2.35
```

```

# Two-way interaction plots
# 1. Stack x Corruption
group_means_sc <- data %>%
  group_by(stack, corruption) %>%
  summarise(
    mean_value = mean(value),
    sd = sd(value),
    n = n(),
    se = sd / sqrt(n),
    .groups = 'drop'
  )

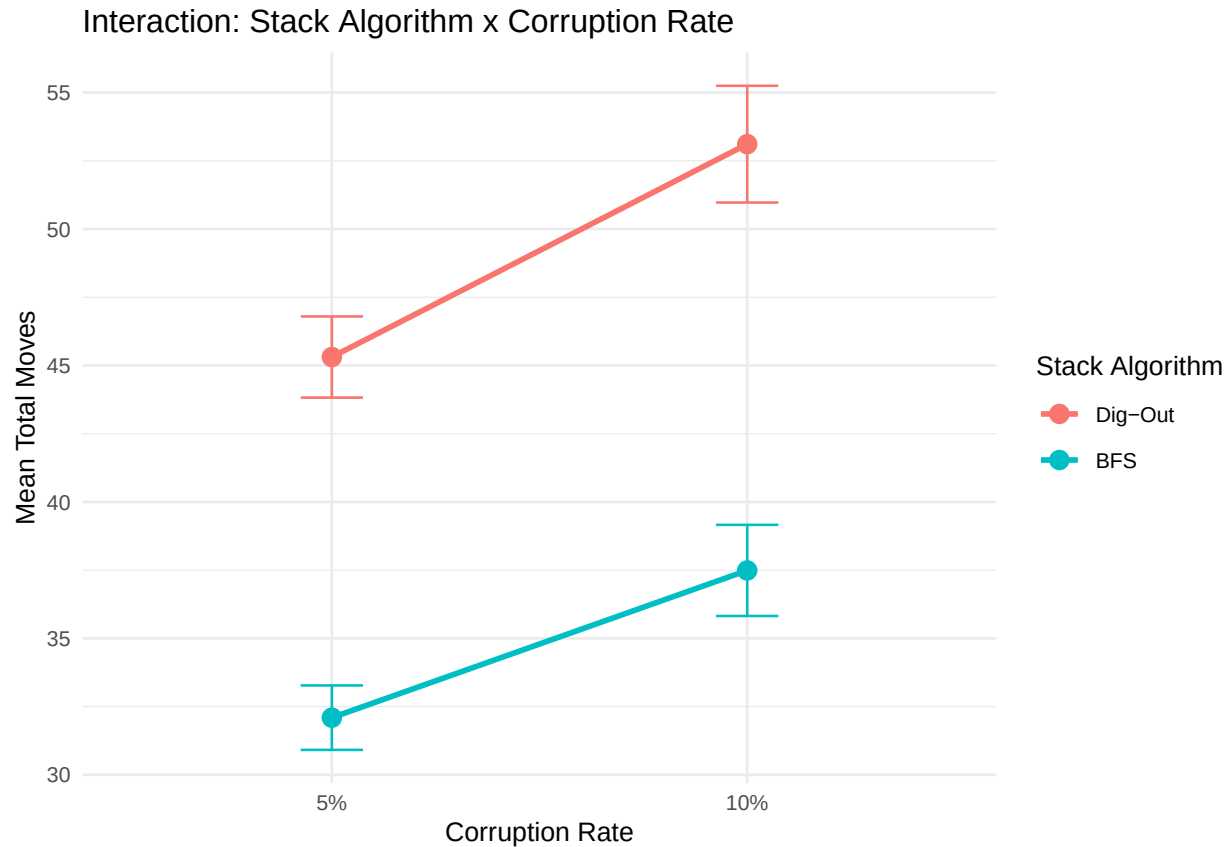
ggplot(group_means_sc, aes(x = corruption, y = mean_value, color = stack, group = stack)) +
  geom_line(size = 1) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = mean_value - se, ymax = mean_value + se), width = 0.15) +
  labs(title = "Interaction: Stack Algorithm x Corruption Rate",
       y = "Mean Total Moves",
       x = "Corruption Rate",
       color = "Stack Algorithm") +
  theme_minimal(base_size = 10)

```

```

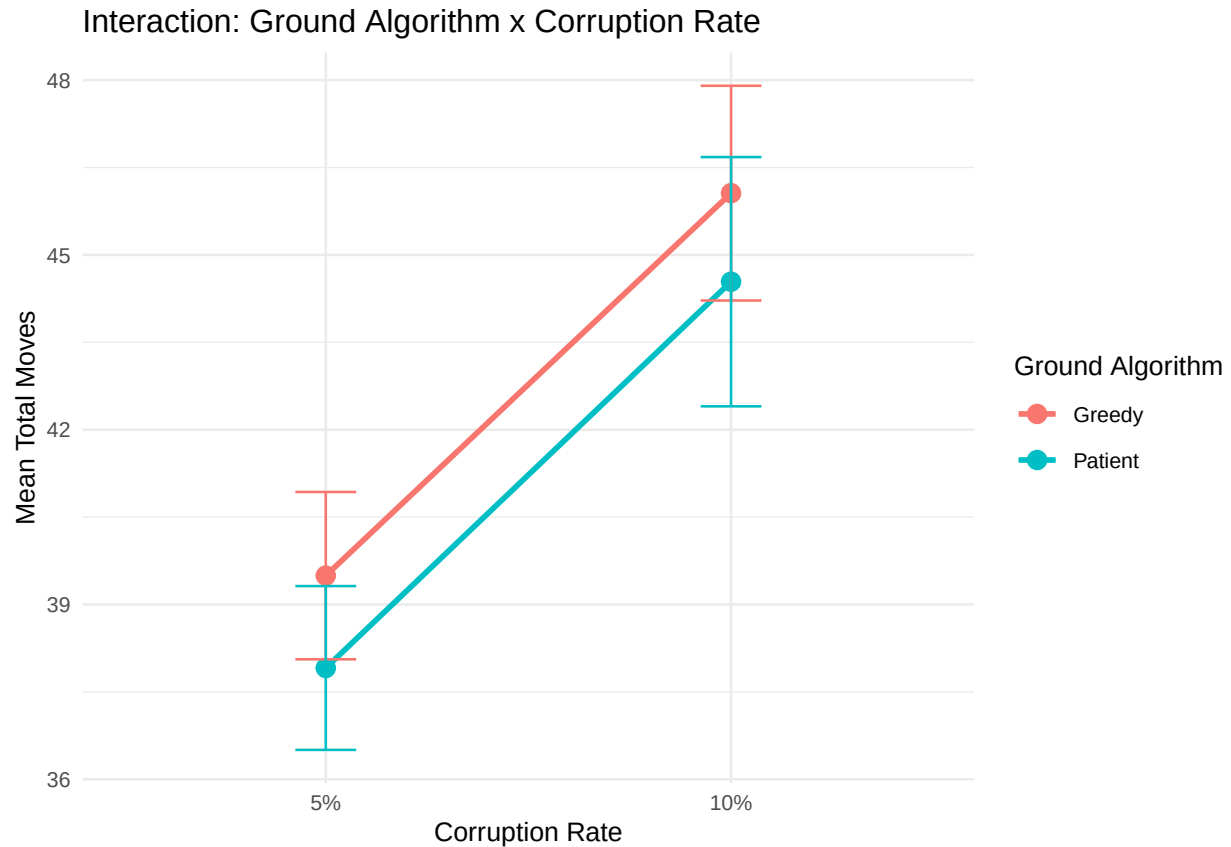
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.

```



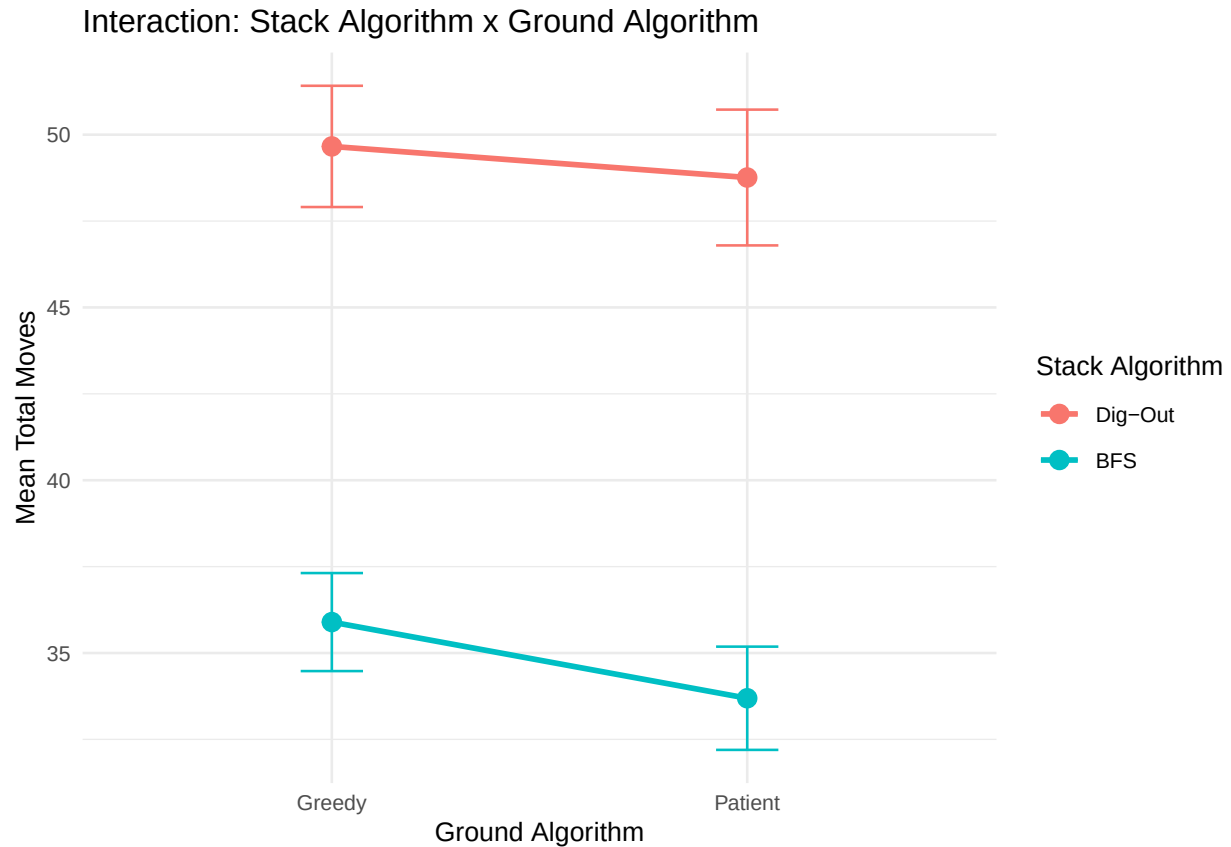
```
# 2. Ground x Corruption
group_means_gc <- data %>%
  group_by(ground, corruption) %>%
  summarise(
    mean_value = mean(value),
    sd = sd(value),
    n = n(),
    se = sd / sqrt(n),
    .groups = 'drop'
  )

ggplot(group_means_gc, aes(x = corruption, y = mean_value, color = ground, group = ground)) +
  geom_line(size = 1) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = mean_value - se, ymax = mean_value + se), width = 0.15) +
  labs(title = "Interaction: Ground Algorithm x Corruption Rate",
       y = "Mean Total Moves",
       x = "Corruption Rate",
       color = "Ground Algorithm") +
  theme_minimal(base_size = 10)
```



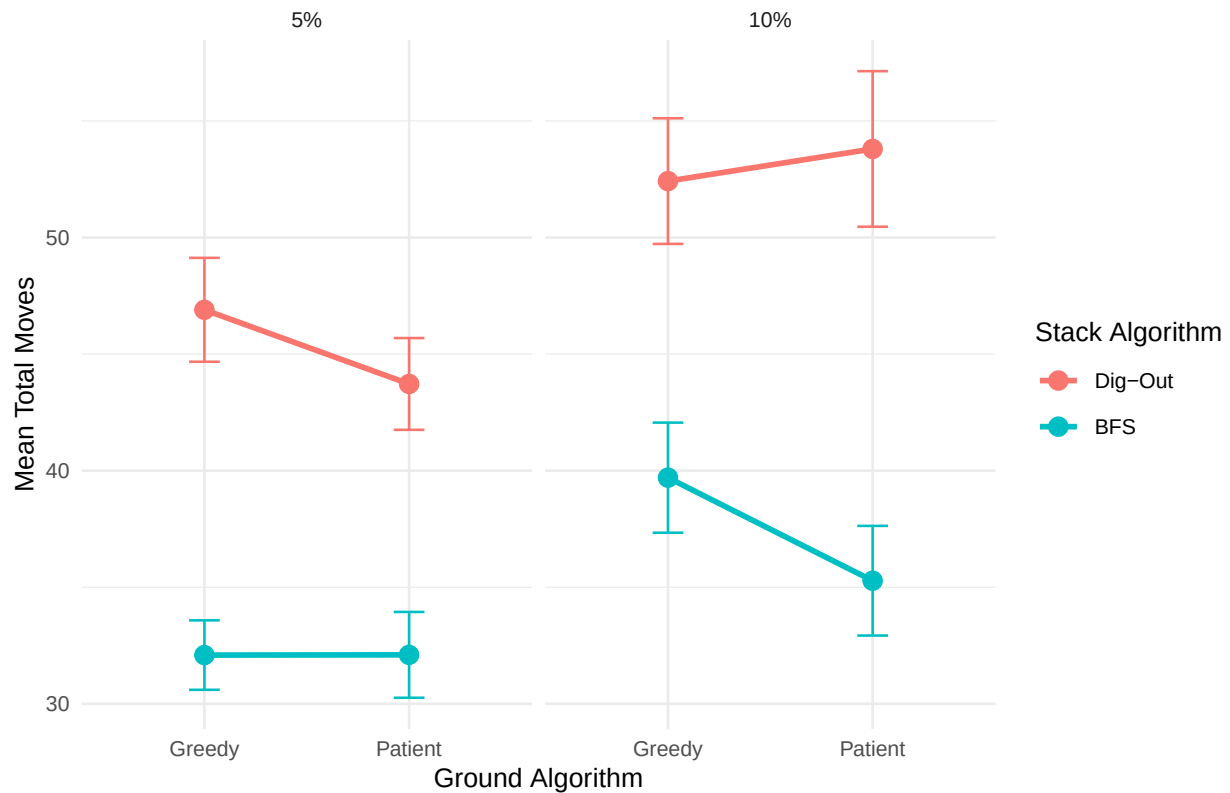
```
# 3. Stack x Ground
group_means_sg <- data %>%
  group_by(stack, ground) %>%
  summarise(
    mean_value = mean(value),
    sd = sd(value),
    n = n(),
    se = sd / sqrt(n),
    .groups = 'drop'
  )

ggplot(group_means_sg, aes(x = ground, y = mean_value, color = stack, group = stack)) +
  geom_line(size = 1) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = mean_value - se, ymax = mean_value + se), width = 0.15) +
  labs(title = "Interaction: Stack Algorithm x Ground Algorithm",
       y = "Mean Total Moves",
       x = "Ground Algorithm",
       color = "Stack Algorithm") +
  theme_minimal(base_size = 10)
```

```
# Three-way interaction plot (Stack x Ground faceted by Corruption)
ggplot(group_means, aes(x = ground, y = mean_value, color = stack, group = stack)) +
  geom_line(size = 1) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = mean_value - se, ymax = mean_value + se), width = 0.15) +
  facet_wrap(~corruption) +
  labs(
    title = "Three-Way Interaction: Stack x Ground x Corruption",
    x = "Ground Algorithm",
    y = "Mean Total Moves",
    color = "Stack Algorithm"
  ) +
  theme_minimal(base_size = 10)
```

Three-Way Interaction: Stack x Ground x Corruption

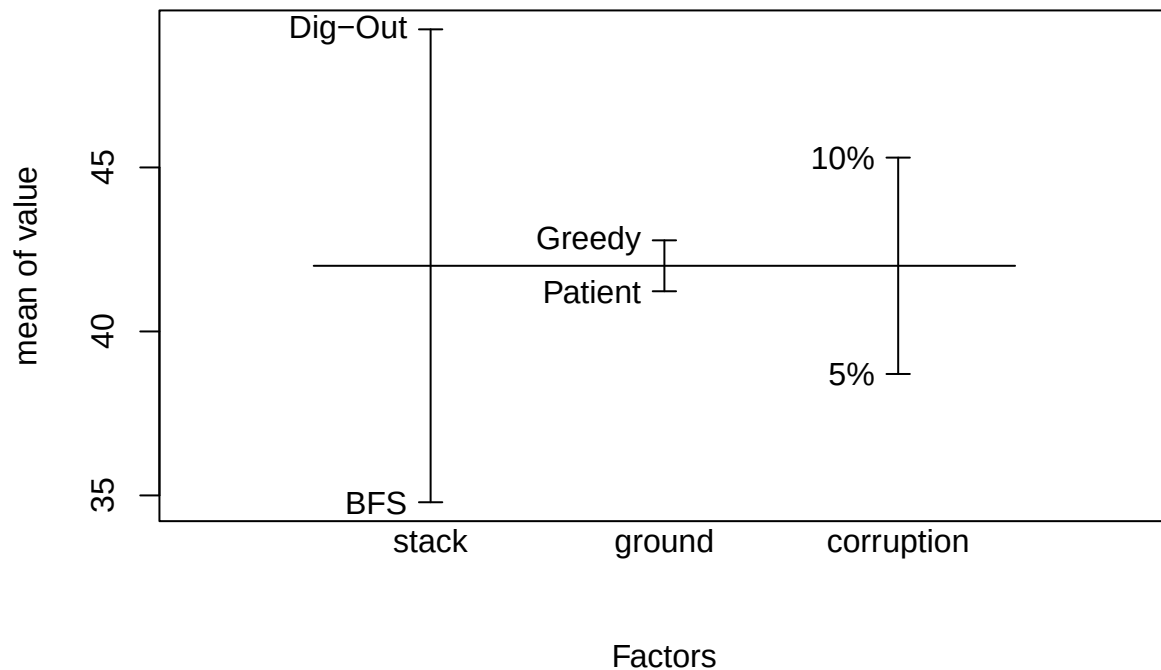


```
# Fit three-way ANOVA model with all interactions
# Model: Total Moves ~ Stack * Ground * Corruption
model <- aov(value ~ stack * ground * corruption, data = data)

# Display ANOVA table with significance tests
summary(model)
```

```
##              Df Sum Sq Mean Sq F value    Pr(>F)
## stack         1  41573    41573  75.689 < 2e-16 ***
## ground        1    482      482   0.878  0.349
## corruption    1   8705    8705  15.849 7.49e-05 ***
## stack:ground   1     85      85   0.155  0.694
## stack:corruption 1    289    289   0.527  0.468
## ground:corruption 1     0      0   0.000  0.984
## stack:ground:corruption 1  1010  1010   1.839  0.175
## Residuals     792 435012    549
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Store results and create effects plot
anova_sum <- summary(model)[[1]]
plot.design(value ~ stack + ground + corruption, data = data)
```



```
# Calculate Observed Effect Size from Data
k <- 8 # Number of groups (2 stack x 2 ground x 2 corruption)
n_per_group <- nrow(data) / k # samples per group

# Calculate eta-squared (eta^2) from ANOVA
ss_total <- sum((data$value - mean(data$value))^2)
ss_residual <- sum(residuals(model)^2)
ss_treatment <- ss_total - ss_residual
eta_squared <- ss_treatment / ss_total

# Convert eta-squared to Cohen's f
# f = sqrt(eta^2 / (1 - eta^2))
f_observed <- sqrt(eta_squared / (1 - eta_squared))

cat("Observed Effect Size:\n")
```

```
## Observed Effect Size:
```

```
cat("  Eta-squared (eta^2):", round(eta_squared, 4), "\n")
```

```
##  Eta-squared (eta^2): 0.107
```

```
cat("  Cohen's f:", round(f_observed, 4), "\n")
```

```
##  Cohen's f: 0.3462
```

```

# Sample Size Calculation (using observed effect size)
pwr_result <- pwr.anova.test(k = k, f = f_observed, sig.level = 0.05, power = 0.9)
cat("\nRequired sample size per group (at observed effect):", ceiling(pwr_result$n), "\n")

##
## Required sample size per group (at observed effect): 20

# Calculate actual power achieved
actual_power <- pwr.anova.test(k = k, n = n_per_group, f = f_observed, sig.level = 0.05)$power
cat("Actual power achieved:", round(actual_power, 4), "\n")

## Actual power achieved: 1

# Tukey's HSD Test
tukey_result <- TukeyHSD(model)
print(tukey_result)

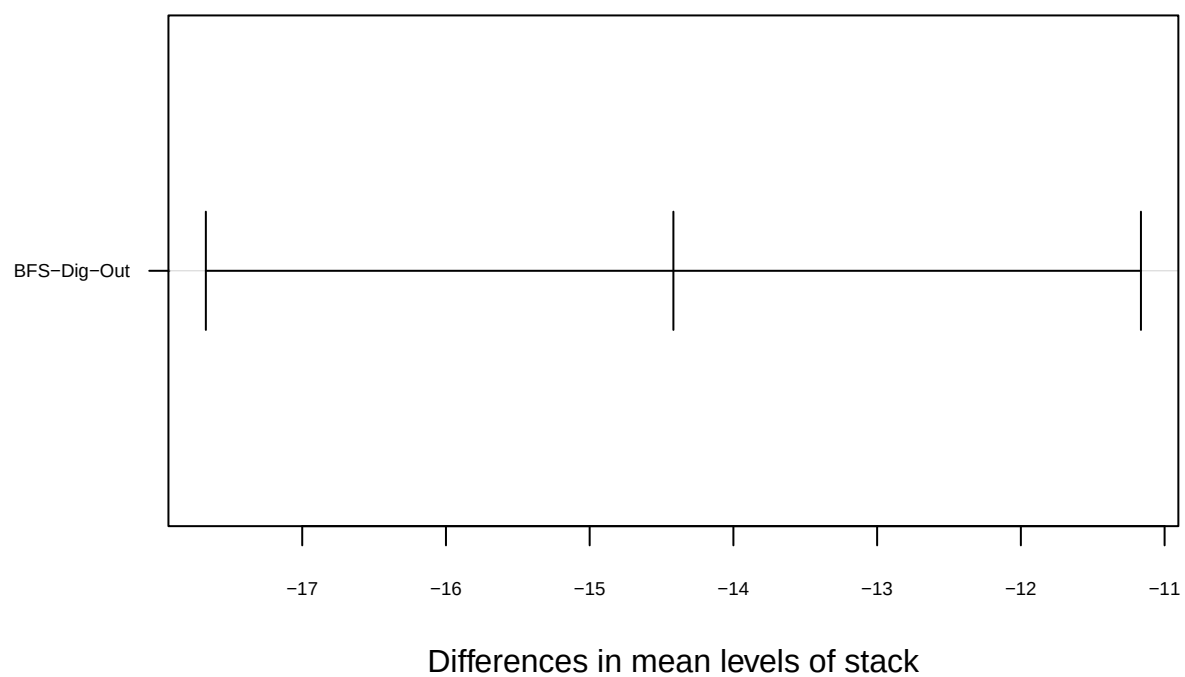
##      Tukey multiple comparisons of means
##      95% family-wise confidence level
##
## Fit: aov(formula = value ~ stack * ground * corruption, data = data)
##
## $stack
##              diff          lwr          upr p adj
## BFS-Dig-Out -14.4175 -17.67051 -11.16449      0
##
## $ground
##              diff          lwr          upr    p adj
## Patient-Greedy -1.5525 -4.805509 1.700509 0.349134
##
## $corruption
##              diff          lwr          upr    p adj
## 10%-5% 6.5975 3.344491 9.850509 7.49e-05
##
## $'stack:ground'
##              diff          lwr          upr    p adj
## BFS:Greedy-Dig-Out:Greedy -13.765 -19.798645 -7.731355 0.0000000
## Dig-Out:Patient-Dig-Out:Greedy -0.900 -6.933645 5.133645 0.9807329
## BFS:Patient-Dig-Out:Greedy -15.970 -22.003645 -9.936355 0.0000000
## Dig-Out:Patient-BFS:Greedy 12.865 6.831355 18.898645 0.0000003
## BFS:Patient-BFS:Greedy -2.205 -8.238645 3.828645 0.7828245
## BFS:Patient-Dig-Out:Patient -15.070 -21.103645 -9.036355 0.0000000
##
## $'stack:corruption'
##              diff          lwr          upr    p adj
## BFS:5%-Dig-Out:5% -13.215 -19.2486454 -7.181355 0.0000001
## Dig-Out:10%-Dig-Out:5% 7.800 1.7663546 13.833645 0.0050544
## BFS:10%-Dig-Out:5% -7.820 -13.8536454 -1.786355 0.0049083
## Dig-Out:10%-BFS:5% 21.015 14.9813546 27.048645 0.0000000
## BFS:10%-BFS:5% 5.395 -0.6386454 11.428645 0.0984127
## BFS:10%-Dig-Out:10% -15.620 -21.6536454 -9.586355 0.0000000
##

```

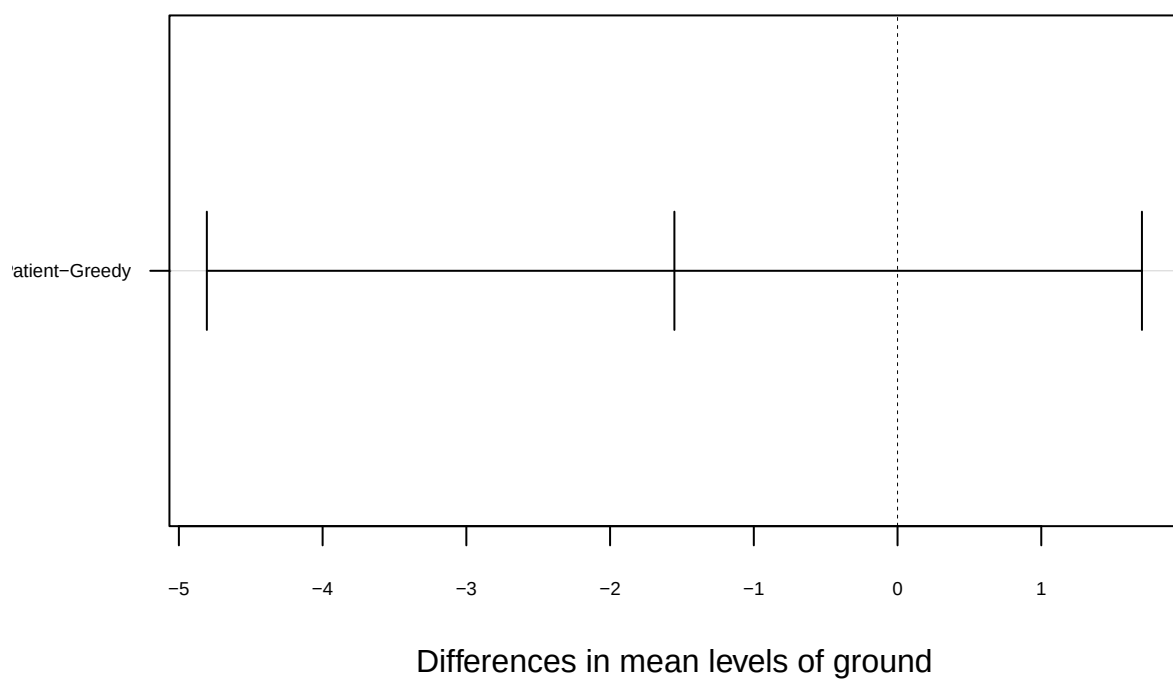
```
## $'ground:corruption'
##               diff          lwr          upr          p adj
## Patient:5%-Greedy:5%   -1.585 -7.6186454  4.448645  0.9060776
## Greedy:10%-Greedy:5%    6.565  0.5313546 12.598645  0.0267383
## Patient:10%-Greedy:5%   5.045 -0.9886454 11.078645  0.1376310
## Greedy:10%-Patient:5%   8.150  2.1163546 14.183645  0.0029928
## Patient:10%-Patient:5%  6.630  0.5963546 12.663645  0.0246730
## Patient:10%-Greedy:10% -1.520 -7.5536454  4.513645  0.9160534
##
## $'stack:ground:corruption'
##               diff          lwr          upr          p adj
## BFS:Greedy:5%-Dig-Out:Greedy:5% -14.81 -24.882532717 -4.737467 0.0002415
## Dig-Out:Patient:5%-Dig-Out:Greedy:5% -3.18 -13.252532717  6.892533 0.9797897
## BFS:Patient:5%-Dig-Out:Greedy:5% -14.80 -24.872532717 -4.727467 0.0002448
## Dig-Out:Greedy:10%-Dig-Out:Greedy:5%  5.52 -4.552532717 15.592533 0.7096633
## BFS:Greedy:10%-Dig-Out:Greedy:5% -7.20 -17.272532717  2.872533 0.3697216
## Dig-Out:Patient:10%-Dig-Out:Greedy:5%  6.90 -3.172532717 16.972533 0.4276259
## BFS:Patient:10%-Dig-Out:Greedy:5% -11.62 -21.692532717 -1.547467 0.0112720
## Dig-Out:Patient:5%-BFS:Greedy:5%  11.63  1.557467283 21.702533 0.0111545
## BFS:Patient:5%-BFS:Greedy:5%  0.01 -10.062532717 10.082533 1.0000000
## Dig-Out:Greedy:10%-BFS:Greedy:5%  20.33 10.257467283 30.402533 0.0000000
## BFS:Greedy:10%-BFS:Greedy:5%  7.61 -2.462532717 17.682533 0.2969712
## Dig-Out:Patient:10%-BFS:Greedy:5%  21.71 11.637467283 31.782533 0.0000000
## BFS:Patient:10%-BFS:Greedy:5%  3.19 -6.882532717 13.262533 0.9794230
## BFS:Patient:5%-Dig-Out:Patient:5% -11.62 -21.692532717 -1.547467 0.0112720
## Dig-Out:Greedy:10%-Dig-Out:Patient:5%  8.70 -1.372532717 18.772533 0.1484774
## BFS:Greedy:10%-Dig-Out:Patient:5% -4.02 -14.092532717  6.052533 0.9281480
## Dig-Out:Patient:10%-Dig-Out:Patient:5% 10.08  0.007467283 20.152533 0.0496740
## BFS:Patient:10%-Dig-Out:Patient:5% -8.44 -18.512532717  1.632533 0.1776917
## Dig-Out:Greedy:10%-BFS:Patient:5%  20.32 10.247467283 30.392533 0.0000000
## BFS:Greedy:10%-BFS:Patient:5%  7.60 -2.472532717 17.672533 0.2986449
## Dig-Out:Patient:10%-BFS:Patient:5%  21.70 11.627467283 31.772533 0.0000000
## BFS:Patient:10%-BFS:Patient:5%  3.18 -6.892532717 13.252533 0.9797897
## BFS:Greedy:10%-Dig-Out:Greedy:10% -12.72 -22.792532717 -2.647467 0.0033492
## Dig-Out:Patient:10%-Dig-Out:Greedy:10%  1.38 -8.692532717 11.452533 0.9999006
## BFS:Patient:10%-Dig-Out:Greedy:10% -17.14 -27.212532717 -7.067467 0.0000081
## Dig-Out:Patient:10%-BFS:Greedy:10%  14.10  4.027467283 24.172533 0.0006176
## BFS:Patient:10%-BFS:Greedy:10% -4.42 -14.492532717  5.652533 0.8858937
## BFS:Patient:10%-Dig-Out:Patient:10% -18.52 -28.592532717 -8.447467 0.0000009
```

```
plot(tukey_result, las = 1, cex.axis = 0.6)
```

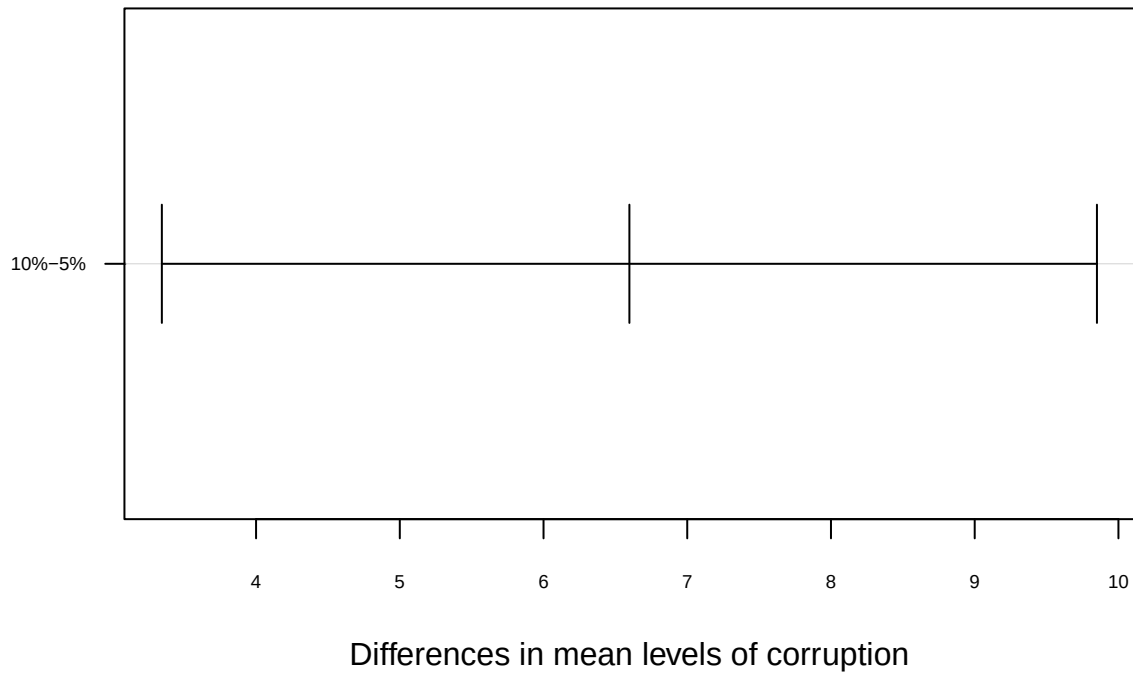
95% family-wise confidence level



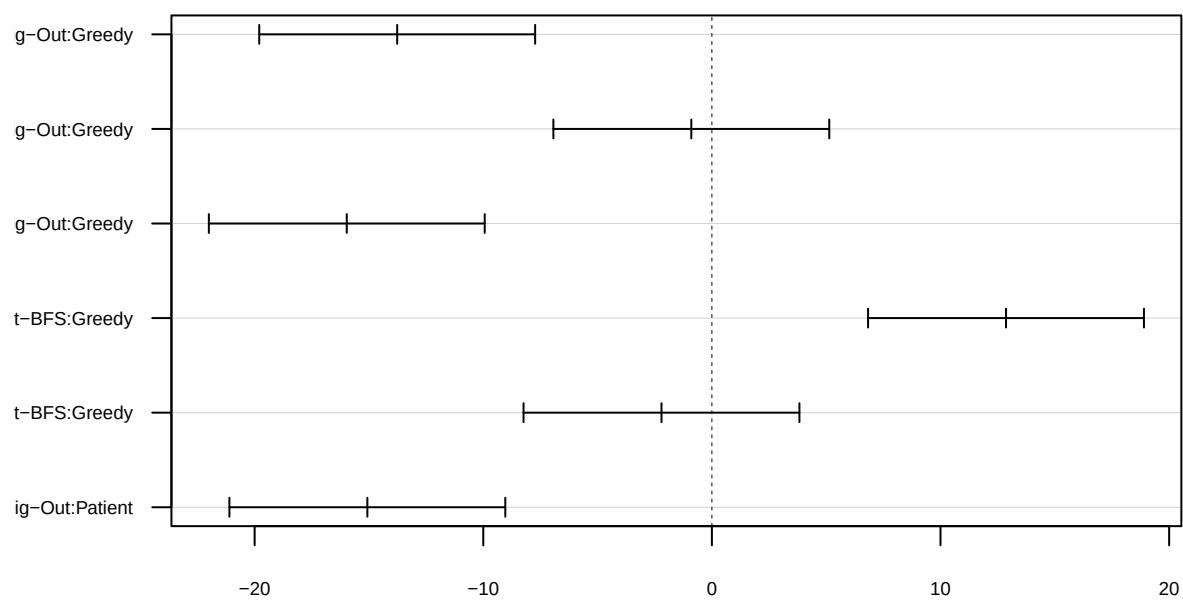
95% family-wise confidence level



95% family-wise confidence level

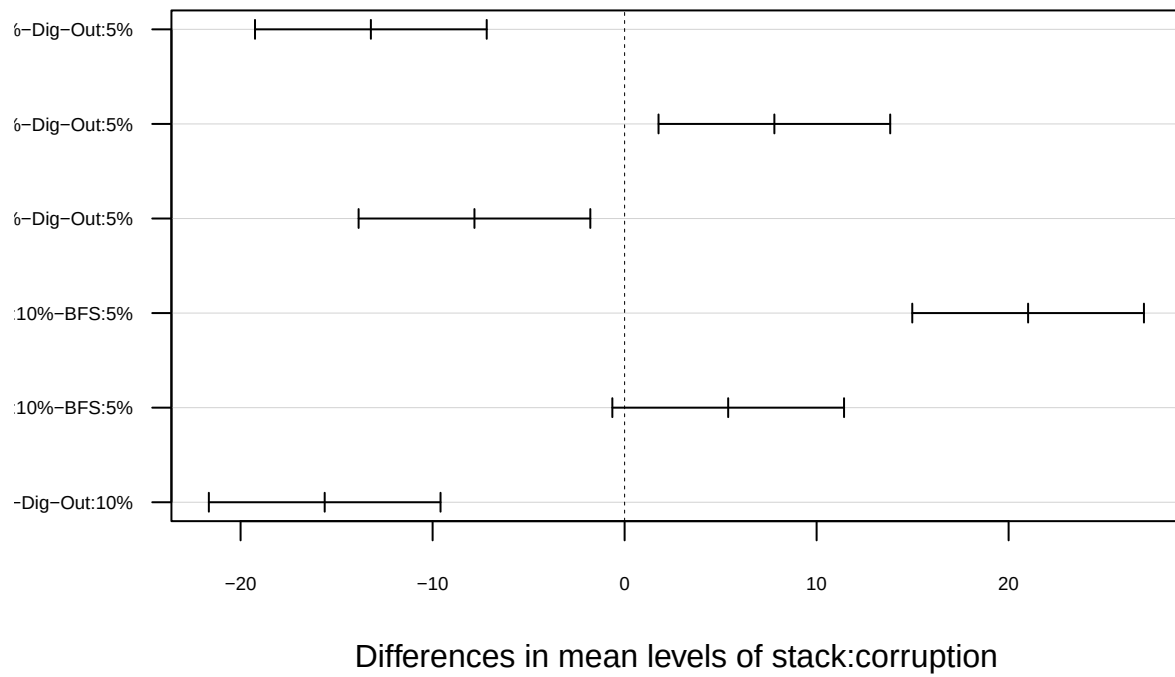


95% family-wise confidence level

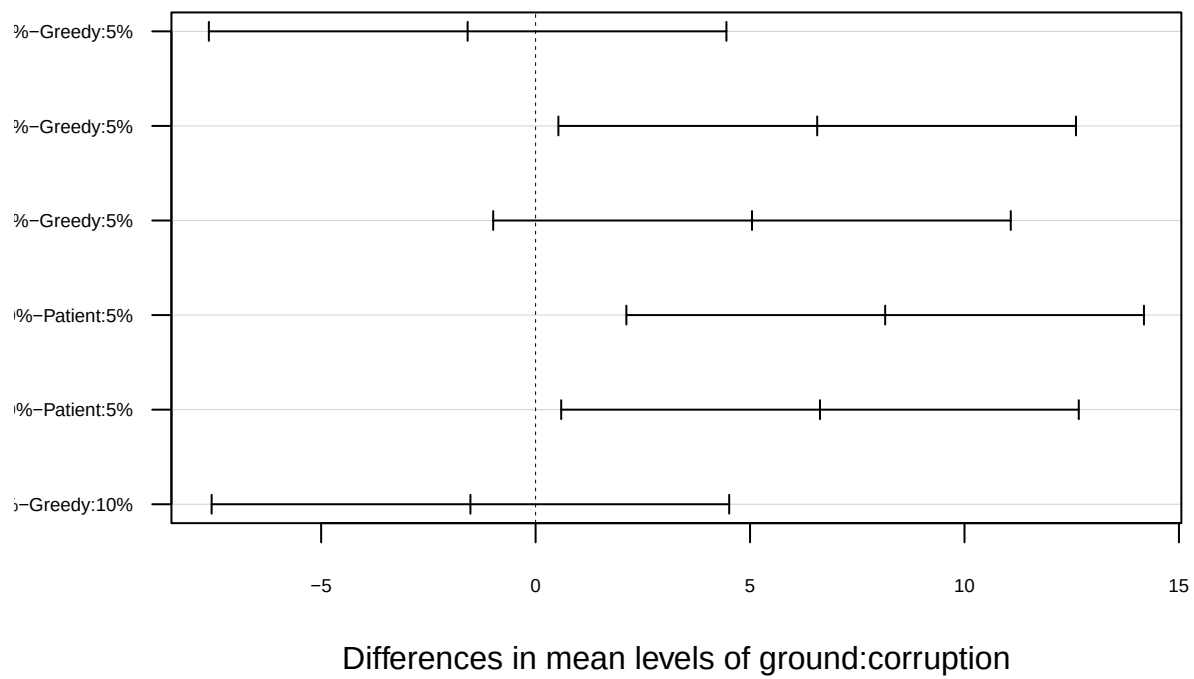


Differences in mean levels of stack:ground

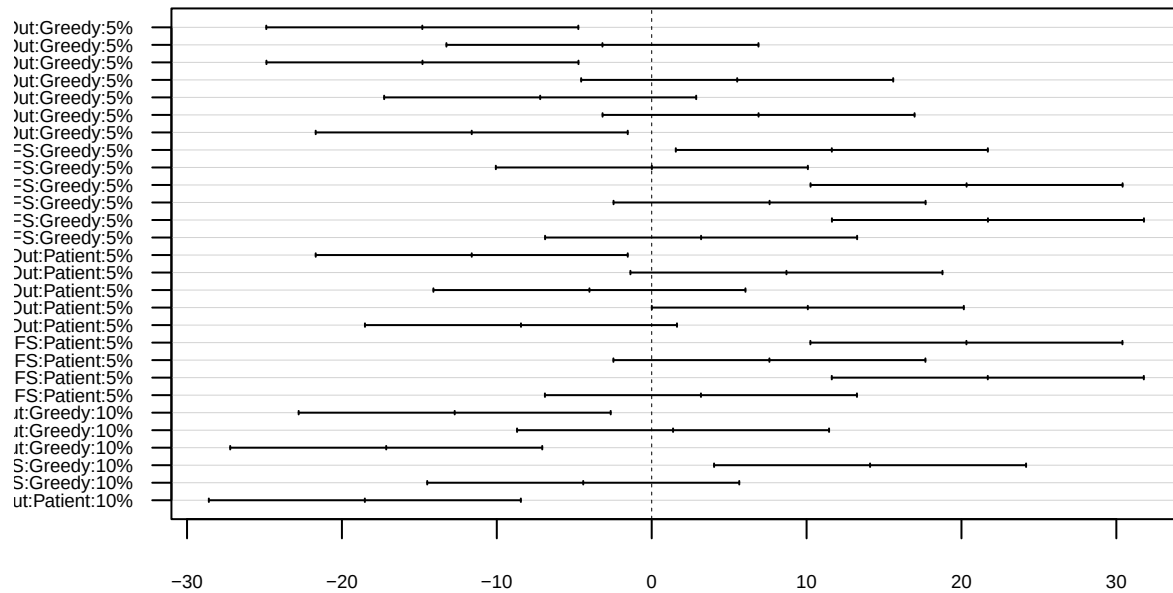
95% family-wise confidence level



95% family-wise confidence level



95% family-wise confidence level



Differences in mean levels of stack:ground:corruption

```
# Estimated Marginal Means (EMMs) - Proper contrasts respecting interactions
cat("\n=== Estimated Marginal Means Analysis ===\n\n")
```

```
##
## === Estimated Marginal Means Analysis ===
```

```
# Main effects
emm_stack <- emmeans(model, ~ stack)
```

```
## NOTE: Results may be misleading due to involvement in interactions
```

```
emm_ground <- emmeans(model, ~ ground)
```

```
## NOTE: Results may be misleading due to involvement in interactions
```

```
emm_corruption <- emmeans(model, ~ corruption)
```

```
## NOTE: Results may be misleading due to involvement in interactions
```

```
cat("Main Effect: Stack Algorithm\n")
```

```
## Main Effect: Stack Algorithm
```

```
print(emm_stack)
```

```
##  stack    emmean    SE  df lower.CL upper.CL
##  Dig-Out   49.2 1.17 792    46.9    51.5
##  BFS       34.8 1.17 792    32.5    37.1
##
## Results are averaged over the levels of: ground, corruption
## Confidence level used: 0.95
```

```
print(pairs(emm_stack))
```

```
##  contrast          estimate    SE  df t.ratio p.value
##  (Dig-Out) - BFS      14.4 1.66 792   8.700 <.0001
##
## Results are averaged over the levels of: ground, corruption
```

```
cat("\nMain Effect: Ground Algorithm\n")
```

```
##
## Main Effect: Ground Algorithm
```

```
print(emm_ground)
```

```
##  ground  emmean    SE  df lower.CL upper.CL
##  Greedy   42.8 1.17 792    40.5    45.1
##  Patient  41.2 1.17 792    38.9    43.5
##
## Results are averaged over the levels of: stack, corruption
## Confidence level used: 0.95
```

```
print(pairs(emm_ground))
```

```
##  contrast          estimate    SE  df t.ratio p.value
##  Greedy - Patient      1.55 1.66 792   0.937 0.3491
##
## Results are averaged over the levels of: stack, corruption
```

```
cat("\nMain Effect: Corruption Rate\n")
```

```
##
## Main Effect: Corruption Rate
```

```
print(emm_corruption)
```

```
##  corruption emmean    SE  df lower.CL upper.CL
##  5%          38.7 1.17 792    36.4    41.0
##  10%         45.3 1.17 792    43.0    47.6
##
## Results are averaged over the levels of: stack, ground
## Confidence level used: 0.95
```

```
print(pairs(emm_corruption))
```

```
## contrast estimate SE df t.ratio p.value
## 5% - 10% -6.6 1.66 792 -3.981 0.0001
##
## Results are averaged over the levels of: stack, ground
```

```
# Two-way interactions
cat("\n=== Two-Way Interactions ===\n")
```

```
##
## === Two-Way Interactions ===
```

```
emm_stack_ground <- emmeans(model, ~ stack | ground)
```

```
## NOTE: Results may be misleading due to involvement in interactions
```

```
cat("\nStack x Ground Interaction:\n")
```

```
##
## Stack x Ground Interaction:
```

```
print(emm_stack_ground)
```

```
## ground = Greedy:
## stack emmean SE df lower.CL upper.CL
## Dig-Out 49.7 1.66 792 46.4 52.9
## BFS 35.9 1.66 792 32.6 39.1
##
## ground = Patient:
## stack emmean SE df lower.CL upper.CL
## Dig-Out 48.8 1.66 792 45.5 52.0
## BFS 33.7 1.66 792 30.4 36.9
##
## Results are averaged over the levels of: corruption
## Confidence level used: 0.95
```

```
print(pairs(emm_stack_ground))
```

```
## ground = Greedy:
## contrast estimate SE df t.ratio p.value
## (Dig-Out) - BFS 13.8 2.34 792 5.873 <.0001
##
## ground = Patient:
## contrast estimate SE df t.ratio p.value
## (Dig-Out) - BFS 15.1 2.34 792 6.430 <.0001
##
## Results are averaged over the levels of: corruption
```

```
emm_stack_corruption <- emmeans(model, ~ stack | corruption)
```

```
## NOTE: Results may be misleading due to involvement in interactions
```

```
cat("\nStack x Corruption Interaction:\n")
```

```
##
```

```
## Stack x Corruption Interaction:
```

```
print(emm_stack_corruption)
```

```
## corruption = 5%:
```

```
## stack    emmean    SE  df lower.CL upper.CL
## Dig-Out  45.3 1.66 792    42.1    48.6
## BFS      32.1 1.66 792    28.8    35.3
##
```

```
## corruption = 10%:
```

```
## stack    emmean    SE  df lower.CL upper.CL
## Dig-Out  53.1 1.66 792    49.9    56.4
## BFS      37.5 1.66 792    34.2    40.7
##
```

```
## Results are averaged over the levels of: ground
```

```
## Confidence level used: 0.95
```

```
print(pairs(emm_stack_corruption))
```

```
## corruption = 5%:
```

```
## contrast      estimate    SE  df t.ratio p.value
## (Dig-Out) - BFS    13.2 2.34 792    5.639 <.0001
##
```

```
## corruption = 10%:
```

```
## contrast      estimate    SE  df t.ratio p.value
## (Dig-Out) - BFS    15.6 2.34 792    6.665 <.0001
##
```

```
## Results are averaged over the levels of: ground
```

```
# Three-way interaction
```

```
emm_three_way <- emmeans(model, ~ stack * ground * corruption)
```

```
cat("\nThree-Way Interaction (All 8 Conditions):\n")
```

```
##
```

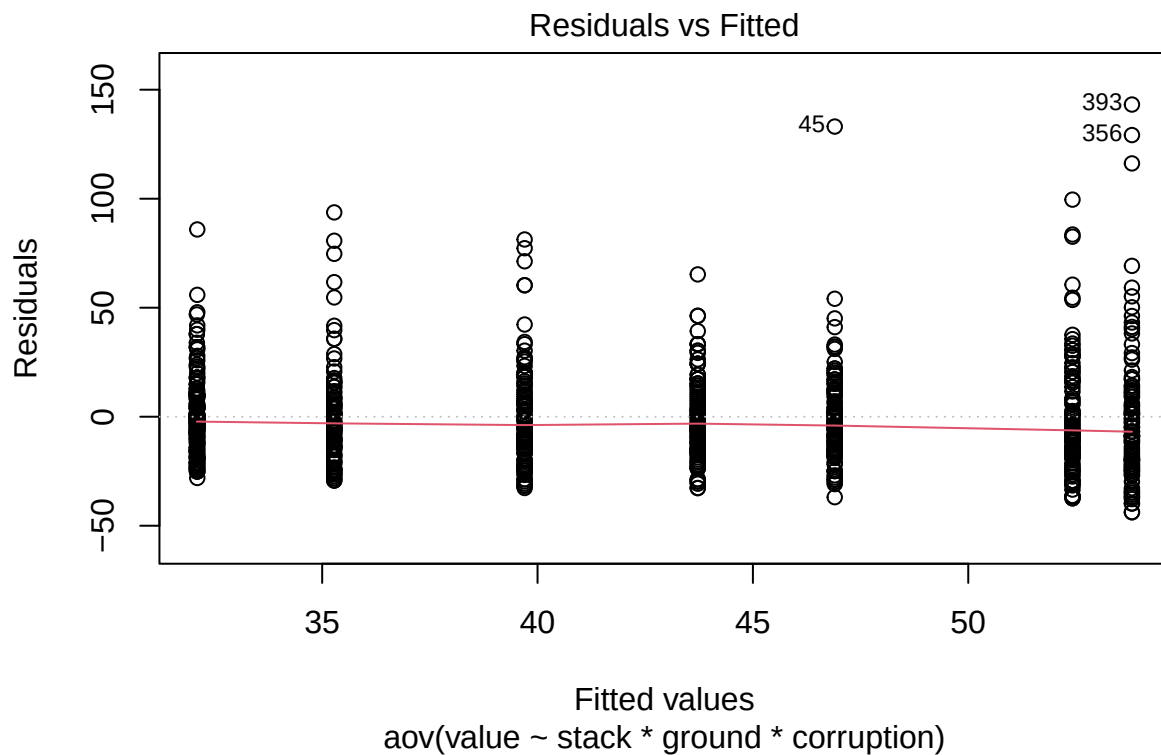
```
## Three-Way Interaction (All 8 Conditions):
```

```
print(emm_three_way)
```

```
## stack    ground  corruption emmean    SE  df lower.CL upper.CL
## Dig-Out Greedy  5%          46.9 2.34 792    42.3    51.5
## BFS      Greedy  5%          32.1 2.34 792    27.5    36.7
## Dig-Out Patient 5%          43.7 2.34 792    39.1    48.3
```

```
## BFS      Patient 5%          32.1 2.34 792      27.5      36.7
## Dig-Out Greedy 10%          52.4 2.34 792      47.8      57.0
## BFS      Greedy 10%          39.7 2.34 792      35.1      44.3
## Dig-Out Patient 10%         53.8 2.34 792      49.2      58.4
## BFS      Patient 10%         35.3 2.34 792      30.7      39.9
##
## Confidence level used: 0.95
```

```
# Assumption Tests
## Residuals vs Fitted
plot(model, which=1)
```



```
## Q-Q Residuals
plot(model, which=2)
```